

CS 236: Operating Systems Lab

Spring 2025-26

Lab 10: futex is fun or is futex fun?

xv6 is a teaching operating system (developed at MIT) — an OS meant for teaching design and implementation concepts of an OS.

To install xv6 on Ubuntu, look up the details described in **Appendix A**.

The Holy Grail for xv6

- [xv6-risc book](#)
- [xv6-risc src booklet](#)

The xv6 source tree is organized to mirror OS subsystems:

- **kernel/** → process management, memory, file system, traps
- **user/** → user programs
- **mkfs/** → file system creation tool

Each directory corresponds to a logical OS component, enabling systematic exploration and modification. **You will mostly interact with the **kernel/** and **user/** directory.**

Submission Guidelines

Create a patch with:

```
Shell
> ./patch.sh
```

Rename the generated **lab10.patch** to **<rollno>_lab10.patch** and submit via Moodle.

1. futex

In a multithreaded system, multiple threads often access shared data concurrently. If such accesses are not properly synchronized, it can lead to race conditions, where the final result depends on the timing of the execution rather than the intended logic. To avoid such issues different types of lock-based synchronization primitives are used.

In this assignment, we implement a simplified synchronization mechanism using a **futex**-like interface for user space threads (and possibly processes with shared memory setups). The goal is to understand the role of the kernel to support user space threads and synchronization.

Implement the following user-space **function**:

futex(int *uaddr, int ops, int val)

- The **uaddr** argument points to the futex variable, i.e., the shared memory location used for synchronization.
- **ops** specifies the type of operation
 - 0 (FUTEX_LOCK)
 - 1 (FUTEX_UNLOCK)
- **val** is an operation specific value and is used for the specified operation

Supported operations:

- FUTEX_LOCK
 - **val = 0**: Acquire the lock using spinning (spinlock behavior)
 - **val = 1**: Acquire the lock, but sleep using **futex_wait** if it is not immediately available and retry until success
- FUTEX_UNLOCK
 - Release the lock
 - If **val = 0**: Do not wake up the threads
 - If **val = 1**: Wake up threads waiting on this lock using **futex_wake**

Implement the following system calls that will be used in the **futex** function:

1. **futex_wait(int* uaddr, int val)**
puts the thread to sleep if the value at **uaddr** matches **val** (checks it atomically)
2. **futex_wake(int* uaddr)**
wakes up all the threads waiting on the same address

Hint:

1. Implementation of the **clone** and **join** system calls for xv6 threads is provided as part of the lab.
2. Refer to `spinlock.c`, `spinlock.h`, `sleeplock.c`, `sleeplock.h`
3. **futex** is user space function you may want to implement it in **ulib.c**
4. **futex** does several synchronization related operations in user space — lock checking, issuing system calls for sleep and wakeup, re-checking of locks, spinning for lock etc.

Test your spinlock implementation using **p1.c**. Set CPUS=4 for the following.

```
None
$ p1
Calling Process Print VAR value: 0, N: 4
[T-0] 250000/1000000 iterations done
[T-0] 500000/1000000 iterations done
[T-1] 250000/1000000 iterations done
[T-3] 250000/1000000 iterations done
[T-2] 250000/1000000 iterations done
[T-1] 500000/1000000 iterations done
[T-3] 500000/1000000 iterations done
[T-0] 750000/1000000 iterations done
[T-2] 500000/1000000 iterations done
[T-0] Finished
[T-3] 750000/1000000 iterations done
[T-1] 750000/1000000 iterations done
[T-2] 750000/1000000 iterations done
[T-3] Finished
[T-1] Finished
[T-2] Finished
All threads joined, VAR value: 4000000, expected: 4000000
```

Test the correctness of your sleeplock implementation using **p2.c**.

```
None
$ p2
Calling Process Print VAR value: 0, N: 4
[T-2] 10000/100000 iterations done
[T-0] 10000/100000 iterations done
[T-1] 10000/100000 iterations done
[T-3] 10000/100000 iterations done
[T-0] 20000/100000 iterations done
[T-2] 20000/100000 iterations done
...
...
[T-3] 90000/100000 iterations done
[T-0] Finished
[T-1] Finished
[T-2] Finished
[T-3] Finished
All threads joined, VAR value: 400000, expected: 400000
```

Notice that sleeplocks are much slower than spinlocks for small critical sections.

Now, run **p3.c** with both types of locks and observe the cpu usage using any monitoring tool such as **top**. Notice that sleeplocks are much better for larger critical section blocks.

```
None
$ p3
[T-0] in CS
[T-1] in CS
[T-2] in CS
[T-3] in CS
[T-0] in CS
[T-3] in CS
[T-2] in CS
[T-1] in CS
[T-3] in CS
[T-1] in CS
[T-0] in CS
[T-2] in CS
[T-3] in CS
[T-0] in CS
```

2. futex-enabled semaphores

As part of this question, we implement a simple **semaphore API** in user space using the **futex** interface. The semaphore API uses the **futex** function to block and wake threads efficiently.

A semaphore is represented as an integer counter:

```
typedef struct {
    int value;
    int lock; // for use with futex
} sem_t;
```

Implement the following functions:

- **sem_init(sem_t* s, int val):**
Initializes the semaphore with a given value. This value represents the number of available resources.
- **sem_down(sem_t* s):**
Attempts to decrement the semaphore. If the value is greater than zero, the thread

proceeds after decrementing it. Otherwise, the thread blocks using **futex_wait** until the value becomes positive.

- **sem_up(sem_t* s):**
Increments the semaphore value and wakes up all waiting threads (if any) using **futex_wake**.

Note:

1. All operations (of the semaphore) that use the value of the semaphore will need mutually exclusive access (including the **init** function).
2. No busy waiting by relying on futex-based blocking within the semaphore implementation.

Write a user-space program **p4.c** that uses the semaphore API implemented in the previous part to enforce ordered execution among threads.

The program should create three threads, each with a different function. Each thread prints a message of the form **T-<id>: hello** and then uses the semaphore API to signal the next thread. The threads must print in this strict order, and the sequence should repeat N times, N is provided as a command-line argument.

The program should:

- Use the semaphore API (**sem_init**, **sem_down**, **sem_up**) for synchronization.
- Ensure that the ordering is maintained across all iterations.

```
None
$ p4
Usage: p4 N
$ p4 3
T-0: hello
T-1: hello
T-2: hello
T-0: hello
T-1: hello
T-2: hello
T-0: hello
T-1: hello
T-2: hello
```

3. Producer-Consumer (optional)

In a multithreaded system, multiple producer and consumer threads share a finite-size buffer. Producers generate data items and insert them into the buffer, while consumers remove and process these items. Since the buffer is shared, concurrent access without proper synchronization can lead to race conditions, buffer overflows, and inconsistent data.

The task is to implement the Producer-Consumer problem using the semaphore API built on top of the futex interface in xv6.

Design a correct synchronization mechanism using semaphores such that:

- Producers wait when the buffer is full.
- Consumers wait when the buffer is empty.
- Access to the shared buffer is mutually exclusive.
- No busy waiting is used; all blocking must be handled using futex-based sleep/wake.

Implement the following functions in **prod-cons.c**:

- `void produce(int item);`
- `int consume();`

Sample output:

```
Shell
$ prod-cons
Usage: prod-cons <mode>
mode: 0 = 1P1C, 1 = 2P1C, 2 = 3P3C
$ prod-cons 0
P[0]: produced 0
C[0]: consumed 0
P[0]: produced 1
C[0]: consumed 1
P[0]: produced 2
C[0]: consumed 2
P[0]: produced 3
C[0]: consumed 3
$ prod-cons 1
P[0]: produced 0
P[1]: produced 10
P[0]: produced 1
C[0]: consumed 0
C[0]: consumed 10
P[0]: produced 2
```

```
C[0]: consumed 1
P[1]: produced 11
C[0]: consumed 2
P[0]: produced 3
C[0]: consumed 11
P[1]: produced 12
C[0]: consumed 3
P[1]: produced 13
C[0]: consumed 12
C[0]: consumed 13
```

4. Reader-Writer (optional)

In a multithreaded system, a shared resource (for example, a database, file, or memory region) is accessed by two types of threads:

- **Readers:** Only read shared data. Multiple readers can safely read concurrently.
- **Writers:** Modify shared data. A writer must have exclusive access.

Constraints

Design a synchronization mechanism using **semaphores (built on futex)** such that:

1. Multiple readers can read the shared resource simultaneously.
2. Writers must have **exclusive access** (no readers or other writers while writing).
3. If a writer is active, no new readers should enter the critical section.
4. If readers are active, writers must wait.
5. No busy waiting. All blocking must use futex-based sleep/wake via semaphores.

Implement the following functions in **rw.c**:

- `void read_shared_data(int tid);`
- `void write_shared_data();`

Each reader/writer will repeatedly access a shared variable:

- Readers reads the value of **shared_data**
- Writers increment **shared_data**

Sample output:

```
None
$ rw
W: wrote 1
W: wrote 2
R[0]: read 2
R[2]: read 2
R[0]: read 2
R[1]: read 2
R[2]: read 2
R[1]: read 2
R[0]: read 2
R[2]: read 2
R[0]: read 2
R[1]: read 2
R[0]: read 2
R[2]: read 2
R[1]: read 2
R[2]: read 2
R[1]: read 2
W: wrote 3
W: wrote 4
W: wrote 5
Final value of shared_data: 5
```

5. Producer-Consumer and Reader-Writer (Linux pthreads version) (optional)

Re-implement the same **Producer-Consumer** and **Reader-Writer** problems you solved in xv6, but now in a Linux environment using the **pthreads API** instead of your **futex API**.

Use pthread synchronization primitives such as **pthread_mutex_t** and **pthread_cond_t**.

Tasks

1. Implement Producer-Consumer using pthreads in `prod_cons_pthread.c`
2. Implement Reader-Writer synchronization using pthreads in `rw_pthread.c`

Ensure correct synchronization:

- No race conditions
- No busy waiting
- Proper blocking using condition variables

The logic and correctness constraints remain the same as the xv6 versions.

Appendix A

Follow these steps for installing xv6 on a Ubuntu-based Linux machine:

1. Prerequisites

- a. First, update your system:

sudo apt update

- b. Install the required packages:

**sudo apt install -y git build-essential qemu-system-riscv64
gcc-riscv64-unknown-elf**

- **build-essential**

For essential packages like libc, gcc, make, etc.

- **qemu-system-riscv64**

Emulates a RISC-V machine in software.

Your computer's real CPU speaks x86-64, but xv6 is written for RISC-V.

QEMU **pretends to be a RISC-V computer**, so the OS can run even though the hardware is different.

- **gcc-riscv64-unknown-elf**

It runs on your x86-64 machine but produces **RISC-V binaries**, not x86-64 ones.

We need this because xv6 must be compiled for the **CPU architecture that QEMU is emulating**.

2. Verify toolchain installation

riscv64-unknown-elf-gcc --version

qemu-system-riscv64 --version

If both commands work, your environment is ready.

3. Clone the xv6 RISC-V Repository

**git clone <https://github.com/mit-pdos/xv6-riscv.git>
cd xv6-riscv**

4. Build xv6

make

Run the command to build the OS. If compilation succeeds, you are ready to run xv6.

5. Run xv6 on QEMU

Start the xv6 operating system:

make qemu

Appendix B

A list of riscv ISA items and xv6 items that intersect with the xv6 system call implementation —
For details of these and more look up Chapter 4 of the xv6 book.

C/C++

ecall

sret

stvec

sepc

scause

sstatus (SIE and SPP)

sscratch

satp

csrw

a0

TRAMPOLINE

TRAPFRAME

uservec

usertrap

userret

prepare_return

syscall

copyin

copyout

copyinstr

argint

argstr